

GitHub Actions로 시작하는 CI/CD

— 빈 폴더에서 배포까지, 직접 따라 만드는 실습 —

학생용 핸드아웃 (STEP-by-STEP 워크북)

시작하기 전에

이 실습이 다른 튜토리얼과 다른 점

완성된 코드를 받아서 실행만 해보는 실습이 아닙니다. 빈 폴더에서 시작해, 한 단계씩 직접 코드를 입력하고, 단계마다 GitHub에 PR을 올려서 자동 검사가 되는 모습을 직접 봅니다. 마지막엔 자동으로 배포된 본인의 계산기를 인터넷 주소로 열어보게 됩니다.

완성 후 도달할 모습

- GitHub에 본인의 simple-calculator 저장소가 있다.
- main에 머지할 때마다 CI가 자동으로 돌고, CD가 자동으로 배포한다.
- 본인의 계산기가 <https://<USER>.github.io/simple-calculator/> 에서 동작한다.
- 본인의 GHCR에 Docker 이미지가 등록되어 있다.

필요한 도구

- GitHub 계정
- Git 설치
- Node.js 20 이상
- VS Code (또는 다른 에디터)
- 터미널 (macOS/Linux는 Terminal, Windows는 Git Bash 또는 PowerShell)

전체 단계 미리 보기

- STEP 0 · 빈 폴더 + GitHub repo 만들기
- STEP 1 · HTML/CSS + 더하기 + 단위/통합 테스트 + CI 워크플로우 → 첫 PR
- STEP 2 · 빼기 추가 → 두 번째 PR
- STEP 3 · 곱하기 추가 → 세 번째 PR
- STEP 4 · 나누기(0 처리 포함) 추가 → 네 번째 PR
- STEP 5 · Dockerfile + Playwright + CD 워크플로우 → 다섯 번째 PR (배포!)
- STEP 6 · 일부러 깨뜨려서 CI가 막아주는 모습 보기

?? 각 STEP은 곧 하나의 feature 브랜치 = 하나의 PR입니다. 머지 직후 main으로 돌아오는 것을 잊지 마세요.

STEP 0 · 시작점 준비하기

0-1) 시작점 ZIP 받기

강사가 배포한 simple-calculator-starter.zip을 다운로드해서, 작업하고 싶은 폴더(예: 홈 폴더 아래 projects)에 풀어둡니다. 이 ZIP에는 README.md와 .gitignore 두 파일만 들어 있습니다 - 나머지는 우리가 직접 만들 겁니다.

```
cd ~/projects                # 또는 본인이 원하는 폴더
unzip ~/Downloads/simple-calculator-starter.zip -d simple-calculator
cd simple-calculator
ls -la                      # README.md와 .gitignore가 보이면 OK
```

0-2) Git 초기화 + GitHub 저장소 만들기

GitHub 웹에서

- ☐ GitHub에 로그인 → 우측 상단 + → New repository
- ☐ Repository name: simple-calculator
- ☐ Public 선택 (GitHub Pages 무료 배포를 위해 필요)
- ☐ README/.gitignore/license는 추가하지 않음 (이미 직접 만들 것이므로)
- ☐ Create repository 버튼 클릭
- ☐ 표시되는 저장소 URL을 복사 (예: <https://github.com/insang/simple-calculator.git>)

터미널에서

아래 명령은 simple-calculator 폴더 안에서 실행합니다. <YOUR_URL> 부분을 본인의 저장소 URL로 바꾸세요.

```
echo "# simple-calculator" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/ischung/simple-calculator.git
git push -u origin main
```

?? GitHub 웹에서 새로고침하면 README.md와 .gitignore가 올라가 있는 게 보입니다. 이게 우리의 main 출발전입니다.

0-3) Node.js 프로젝트 초기화

아직 main 브랜치에 있는 상태로, package.json을 만듭니다. STEP 1부터는 feature 브랜치에서 작업하지만, package.json은 처음부터 main에 두는 게 깔끔합니다.

```
npm init -y
```

생성된 package.json을 열어 다음과 같이 바꿉니다 (덮어쓰기 OK).

?? package.json

```
{
  "name": "simple-calculator",
  "version": "1.0.0",
  "description": "GitHub Actions CI/CD 학습용 계산기",
  "type": "module",
  "scripts": {
    "start": "npx http-server src -p 8080 -c-1"
  }
}
```

?? "type": "module"은 우리가 ES 모듈(import/export)을 쓰겠다는 선언입니다. scripts는 STEP을 진행하면서 계속 늘어납니다.

main에 커밋합니다.

```
git add package.json
git commit -m "chore: package.json 초기화"
git push
```

STEP 0 체크리스트

- ☐ GitHub에 simple-calculator 저장소가 만들어졌다
- ☐ 로컬 폴더에 README.md, .gitignore, package.json이 있다
- ☐ git push가 성공했고 GitHub 웹에서 main 브랜치에 세 파일이 보인다

STEP 1 · 더하기 + CI 파이프라인

이번 단계가 가장 양이 많습니다. 여기서 만든 골격(HTML/CSS/JS, 테스트 환경, CI 워크플로우)을 STEP 2~5에서 계속 재사용합니다. 차근차근 따라가세요.

1-1) feature 브랜치 만들기

```
git switch main
git pull
git switch -c feature/add
```

1-2) HTML/CSS 골격

src 폴더를 만들고 그 안에 세 파일을 만듭니다.

?? src/index.html

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>간단 계산기</title>
  <link rel="stylesheet" href="style.css" />
</head>
<body>
  <main class="calculator">
    <h1>간단 계산기</h1>
    <div class="display" data-testid="display">0</div>

    <div class="inputs">
      <label>A: <input type="number" id="inputA" data-testid="inputA" step="any" /></label>
      <label>B: <input type="number" id="inputB" data-testid="inputB" step="any" /></label>
    </div>

    <div class="buttons">
      <button type="button" data-op="add" data-testid="btn-add">+ 더하기</button>

      <button type="button" id="clear" data-testid="btn-clear" class="clear">초기화</button>
    </div>

    <button type="button" id="clear" data-testid="btn-clear" class="clear">초기화</button>
  </main>

  <script type="module" src="app.js"></script>
```

```
</body>
</html>
```

?? `data-testid` 속성은 테스트에서 요소를 찾을 때 쓰는 표식입니다. 디자인이 바뀌어도 테스트가 안 깨지도록 도와줍니다.

?? src/style.css

```
* { box-sizing: border-box; }
body { margin: 0; font-family: system-ui, sans-serif; background: #f3f4f6;
  display: flex; justify-content: center; align-items: center; min-height: 100vh; }
.calculator { background: white; padding: 2rem; border-radius: 12px;
  box-shadow: 0 4px 16px rgba(0,0,0,.08); width: 320px; }
h1 { margin: 0 0 1rem 0; font-size: 1.25rem; text-align: center; }
.display { background: #1f2937; color: #f9fafb; font-size: 2rem;
  text-align: right; padding: 1rem; border-radius: 8px;
  margin-bottom: 1rem; min-height: 2.5rem; }
.inputs { display: flex; flex-direction: column; gap: .5rem; margin-bottom: 1rem; }
.inputs label { display: flex; align-items: center; gap: .5rem; }
.inputs input { flex: 1; padding: .5rem; border: 1px solid #d1d5db; border-radius: 6px; }
.buttons { display: grid; grid-template-columns: 1fr 1fr; gap: .5rem; margin-bottom: .5rem; }
.buttons button { padding: .75rem; border: none; border-radius: 6px;
  background: #2563eb; color: white; font-size: 1rem; cursor: pointer; }
.buttons button:hover { background: #1d4ed8; }
.clear { width: 100%; padding: .6rem; border: 1px solid #d1d5db;
  border-radius: 6px; background: #f9fafb; cursor: pointer; }
```

1-3) 계산기 핵심 로직

계산기의 "두뇌"입니다. 순수 함수만 모아두면 단위 테스트가 매우 쉬워집니다.

?? src/calculator.js

```
// 사칙연산 순수 함수 모음 - 같은 입력에 항상 같은 출력, 외부 상태 변경 없음
export function add(a, b) {
  return a + b;
}

// 연산자 문자열을 받아 알맞은 함수를 골라 실행 (STEP 2부터 함수가 추가됨)
export function calculate(operator, a, b) {
  const ops = { add };
  const fn = ops[operator];
  if (!fn) {
    throw new Error(`알 수 없는 연산자: ${operator}`);
  }
  return fn(a, b);
}
```

```
}
```

?? src/app.js

```
import { calculate } from "./calculator.js";

const display = document.querySelector('[data-testid="display"]');
const inputA = document.getElementById("inputA");
const inputB = document.getElementById("inputB");
const clearBtn = document.getElementById("clear");
const opButtons = document.querySelectorAll("[data-op]");

function readNumber(el) {
  const v = parseFloat(el.value);
  return Number.isFinite(v) ? v : 0;
}

opButtons.forEach((btn) => {
  btn.addEventListener("click", () => {
    const a = readNumber(inputA);
    const b = readNumber(inputB);
    try {
      const result = calculate(btn.dataset.op, a, b);
      display.textContent = String(result);
    } catch (err) {
      display.textContent = "에러: " + err.message;
    }
  });
});

clearBtn.addEventListener("click", () => {
  inputA.value = "";
  inputB.value = "";
  display.textContent = "0";
});
```

브라우저로 직접 확인하고 싶으면:

```
npm start # http://localhost:8080 에서 더하기 동작
```

1-4) 테스트 환경 설정

Jest와 babel, eslint 등을 dev 의존성으로 설치합니다.

```
npm install --save-dev jest jest-environment-jsdom \
  babel-jest @babel/core @babel/preset-env \
  eslint globals http-server
```

package.json의 scripts를 확장합니다 (전체를 다음과 같이 바꿉니다).

?? package.json

```
{
  "name": "simple-calculator",
  "version": "1.0.0",
  "description": "GitHub Actions CI/CD 학습용 계산기",
  "type": "module",
  "scripts": {
    "start": "npx http-server src -p 8080 -c-1",
    "lint": "eslint src tests/unit tests/integration",
    "audit": "npm audit --audit-level=high",
    "test:unit": "jest tests/unit",
    "test:integration": "jest tests/integration",
    "test": "jest"
  },
  "devDependencies": { "...": "npm install이 자동으로 채움" }
}
```

△ "devDependencies" 안의 내용은 직접 쓰지 말고, npm install 명령이 자동으로 채워준 그대로 두세요. 위 표시는 그 자리에 자동으로 채워진다는 의미입니다.

그 외 설정 파일 세 개를 만듭니다.

?? babel.config.cjs

```
module.exports = {
  presets: [["@babel/preset-env", { targets: { node: "current" } }]],
};
```

?? jest.config.js

```
/** @type {import('jest').Config} */
export default {
  testEnvironment: "node",
  transform: { "^.+\\.jsx?$": "babel-jest" },
  testPathIgnorePatterns: ["/node_modules/", "/tests/e2e/"],
};
```

?? eslint.config.js

```
import globals from "globals";

export default [
  {
    files: ["src/**/*.js"],
```



```

languageOptions: {
  ecmaVersion: "latest",
  sourceType: "module",
  globals: { ...globals.browser },
},
rules: {
  "no-unused-vars": "warn",
  "no-undef": "error",
},
},
{
  files: ["tests/**/*.js"],
  languageOptions: {
    ecmaVersion: "latest",
    sourceType: "module",
    globals: { ...globals.node, ...globals.jest, ...globals.browser },
  },
  rules: { "no-unused-vars": "warn" },
},
];

```

1-5) 단위 테스트 작성

tests/unit/ 폴더를 만들고 그 안에 테스트 파일을 둡니다.

?? tests/unit/calculator.test.js

```

// 단위 테스트: 함수 하나를 격리해서 테스트
import { add, calculate } from "../../src/calculator.js";

describe("add", () => {
  test("두 양수를 더한다", () => {
    expect(add(2, 3)).toBe(5);
  });
  test("음수도 처리한다", () => {
    expect(add(-2, 5)).toBe(3);
  });
});

describe("calculate", () => {
  test("연산자 문자열로 알맞은 함수를 호출한다", () => {
    expect(calculate("add", 1, 2)).toBe(3);
  });
  test("알 수 없는 연산자는 에러를 던진다", () => {
    expect(() => calculate("modulo", 1, 2)).toThrow("알 수 없는 연산자");
  });
});

```

```
});  
});
```

실행해서 통과하는지 확인합니다.

```
npm run test:unit
```

?? PASS 메시지와 함께 4개 테스트가 통과해야 합니다.

1-6) 통합 테스트 작성

tests/integration/ 폴더를 만들고 그 안에 테스트 파일을 둡니다. 이 테스트는 진짜 브라우저는 안 띄우지만, jsdom으로 HTML+JS를 흉내 내서 버튼 클릭 → 계산 → display 갱신 흐름을 검증합니다.

?? tests/integration/ui.test.js

```
/**  
 * @jest-environment jsdom  
 */  
import fs from "fs";  
import path from "path";  
  
describe("계산기 UI 통합", () => {  
  beforeEach(async () => {  
    const html = fs.readFileSync(  
      path.resolve(__dirname, "../../src/index.html"), "utf8"  
    );  
    document.documentElement.innerHTML = html;  
    jest.resetModules();  
    await import("../../src/app.js");  
  });  
  
  test("더하기 버튼이 두 입력값을 더해 display에 표시한다", () => {  
    document.getElementById("inputA").value = "7";  
    document.getElementById("inputB").value = "8";  
    document.querySelector("[data-testid='btn-add']").click();  
    expect(document.querySelector("[data-testid='display']").textContent).toBe("15");  
  });  
  
  test("초기화 버튼이 입력과 display를 리셋한다", () => {  
    document.getElementById("inputA").value = "1";  
    document.getElementById("inputB").value = "2";  
    document.querySelector("[data-testid='btn-add']").click();  
    document.querySelector("[data-testid='btn-clear']").click();  
    expect(document.getElementById("inputA").value).toBe("");  
  });  
});
```

```

    expect(document.querySelector('[data-testid="display"]').textContent).toBe("0");
  });
});

npm run test:integration

```

?? PASS 2 tests가 보여야 합니다.

1-7) CI 워크플로우 작성

드디어 CI를 만듭니다. `.github/workflows/` 디렉터리를 만들고 그 안에 `ci.yml`을 둡니다 (정확한 경로 중요!).

?? `.github/workflows/ci.yml`

```

# CI - PR이 열릴 때, 그리고 main에 push될 때 자동으로 실행
name: CI

```

```

on:
  pull_request:
    branches: [main]
  push:
    branches: [main]

jobs:
  lint:
    name: 1) 린트 검사 (ESLint)
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: "20"
          cache: "npm"
      - run: npm ci
      - run: npm run lint

```

```

security:
  name: 2) 보안 검사 (npm audit)
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with:
        node-version: "20"
        cache: "npm"

```

```

- run: npm ci
- run: npm run audit

unit-test:
  name: 3) 단위 테스트 (Jest)
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with:
        node-version: "20"
        cache: "npm"
    - run: npm ci
    - run: npm run test:unit

integration-test:
  name: 4) 통합 테스트 (Jest + jsdom)
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with:
        node-version: "20"
        cache: "npm"
    - run: npm ci
    - run: npm run test:integration

```

1-8) 커밋 → 푸시 → PR 생성

이제 모든 것을 add하고 첫 PR을 올립니다.

```

git add .
git commit -m "feat: 더하기 연산 + 테스트 환경 + CI 파이프라인"
git push -u origin feature/add

```

GitHub 웹으로 가면 노란 배너에 "Compare & pull request" 버튼이 나타납니다. 클릭 → 제목/설명 작성 → Create pull request.

1-9) CI가 도는 모습 보기

PR 페이지를 새로고침하면, 잠시 후 4개 잡(lint/security/unit-test/integration-test)이 노란색으로 보였다가 차례로 초록색 ✓로 바뀝니다. Details 링크를 눌러 실제 로그를 한번 들여다보세요
- runs-on의 Ubuntu가 어떤 명령을 실행하는지 보입니다.

?? 첫 실행은 의존성을 처음 설치하느라 1~2분 정도 걸립니다. 이후 실행은 cache 덕분에 훨씬

빨라집니다.

1-10) Squash merge

- ☐ CI 4개 잡이 모두 초록색이 됐는지 확인
- ☐ PR 페이지에서 Squash and merge 버튼 클릭 → Confirm
- ☐ Delete branch 버튼이 보이면 클릭 (정리)

로컬에서도 정리합니다.

```
git switch main
git pull
git branch -d feature/add
```

STEP 1 체크리스트

- ☐ feature/add 브랜치를 만들어 작업했다
- ☐ npm run test:unit 로컬 통과 (4 tests)
- ☐ npm run test:integration 로컬 통과 (2 tests)
- ☐ PR을 올렸다
- ☐ CI 4개 잡이 모두 초록색이 됐다
- ☐ Squash merge 했다
- ☐ main으로 돌아오고 feature 브랜치를 삭제했다

STEP 2 · 빼기 추가

STEP 1에서 만든 골격 위에서 함수 한 개와 테스트 몇 개만 추가합니다. 사이클 자체에 익숙해지는 것이 목표입니다.

2-1) feature 브랜치

```
git switch main
git pull
git switch -c feature/subtract
```

2-2) calculator.js 수정

subtract 함수를 추가하고, calculate의 ops 객체에도 등록합니다.

?? src/calculator.js

```
export function add(a, b) {
  return a + b;
}

export function subtract(a, b) {
  return a - b;
}

export function calculate(operator, a, b) {
  const ops = { add, subtract };
  const fn = ops[operator];
  if (!fn) {
    throw new Error(`알 수 없는 연산자: ${operator}`);
  }
  return fn(a, b);
}
```

2-3) index.html에 빼기 버튼 추가

.buttons div 안의 더하기 버튼 다음 줄에 추가합니다.

```
<button type="button" data-op="subtract" data-testid="btn-subtract"> - 빼기</button>
```

2-4) 단위 및 통합 테스트 추가

tests/unit/calculator.test.js에 import 줄을 수정하고 describe 블록을 추가합니다.

```
import { add, subtract, calculate } from "../../src/calculator.js";

// (기존 describe들 그대로 두고, 아래 블록만 추가)

describe("subtract", () => {
  test("두 수를 뺀다", () => {
    expect(subtract(10, 4)).toBe(6);
  });
  test("음수 결과도 처리한다", () => {
    expect(subtract(3, 10)).toBe(-7);
  });
});
```

tests/integration/ui.test.js에

```
test("빼기 버튼이 두 입력값을 빼서 display에 표시한다", () =>{
  document.getElementById("inputA").value = "10";
  document.getElementById("inputB").value = "8";
  document.querySelector('[data-test-id="btn-subtract"]').click();
  expect(document.querySelector('[data-test-id="display"]').textContent).toBe("2");
});
```

2-5) 로컬 검증 → PR

```
npm run test:unit
npm run test:integration
git add .
git commit -m "feat: 빼기 연산 추가"
git push -u origin feature/subtract
```

GitHub에서 PR 생성 → CI 통과 확인 → Squash merge → main pull → branch 삭제.

STEP 3 · 곱하기 추가

STEP 2와 똑같은 사이클입니다. 핵심만 표시합니다.

3-1) feature 브랜치

```
git switch main && git pull && git switch -c feature/multiply
```

3-2) calculator.js 변경

```
export function multiply(a, b) {
  return a * b;
}

// calculate 함수의 ops에도 등록
const ops = { add, subtract, multiply };
```

3-3) index.html에 버튼 추가

```
<button type="button" data-op="multiply" data-testid="btn-multiply">× 곱하기</button>
```

3-4) 단위 및 통합 테스트

```
describe("multiply", () => {
  test("두 수를 곱한다", () => {
    expect(multiply(3, 4)).toBe(12);
  });
  test("0과의 곱은 0이다", () => {
    expect(multiply(7, 0)).toBe(0);
  });
});
```

tests/integration/ui.test.js에

```
test("곱하기 버튼이 두 입력값을 곱해서 display에 표시한다", () =>{
  document.getElementById("inputA").value = "2";
  document.getElementById("inputB").value = "9";
  document.querySelector('[data-testid="btn-multiply"]').click();
  expect(document.querySelector('[data-testid="display"]').textContent).toBe("18");
});
```


3-5) 검증 → PR → Squash merge → main pull → branch 삭제

```
npm run test:unit          # 8 tests로 늘어나야 함
npm run test:integration
git add .
git commit -m "feat: 곱하기 연산 추가"
git push -u origin feature/multiply
```

STEP 4 · 나누기 추가 (0 처리 포함)

나누기는 "0으로 나누기" 예외 처리가 들어갑니다. 단위 테스트로 그 동작을 못 박는 좋은 예시이기도 합니다.

4-1) feature 브랜치

```
git switch main && git pull && git switch -c feature/divide
```

4-2) calculator.js 변경

```
export function divide(a, b) {
  if (b === 0) {
    throw new Error("0으로 나눌 수 없습니다");
  }
  return a / b;
}

// ops 등록
const ops = { add, subtract, multiply, divide };
```

4-3) index.html에 버튼 추가

```
<button type="button" data-op="divide" data-testid="btn-divide">÷ 나누기</button>
```

4-4) 단위 테스트

```
describe("divide", () => {
  test("두 수를 나눈다", () => {
    expect(divide(10, 2)).toBe(5);
  });
  test("0으로 나누면 에러를 던진다", () => {
    expect(() => divide(5, 0)).toThrow("0으로 나눌 수 없습니다");
  });
});
```

4-5) 통합 테스트 보강 (선택)

통합 테스트에도 0으로 나누기 동작을 검증하는 테스트를 추가합니다.

```
test("0으로 나누면 에러 메시지를 표시한다", () => {
  document.getElementById("inputA").value = "5";
```

```
document.getElementById("inputB").value = "0";
document.querySelector('[data-testid="btn-divide"]').click();
expect(document.querySelector('[data-testid="display"]').textContent).toMatch(/에러/);
});
```

4-6) 검증 → PR → Squash merge → main 정리

```
npm run test # 단위 + 통합 모두 통과해야 함
```

이 시점에서 본인 GitHub 저장소의 main에는 사칙연산 4개가 모두 들어 있는 계산기가 있고, 로컬에서 npm start로 띄워보면 4개 버튼이 모두 동작합니다. 다음 STEP은 이것 실제로 인터넷에 배포합니다.

STEP 5 · Docker + E2E + CD 파이프라인

지금까지 만든 코드를 자동으로 배포하는 단계입니다. 이번 PR이 머지되는 순간부터, main에 코드가 들어갈 때마다 자동으로 빌드 → 테스트 → 배포가 일어납니다.

5-1) feature 브랜치

```
git switch main && git pull && git switch -c feature/cd
```

5-2) Dockerfile 작성

정적 파일을 nginx로 서빙하는 매우 단순한 이미지입니다.

?? Dockerfile

```
FROM nginx:1.27-alpine

RUN rm -rf /usr/share/nginx/html/*
COPY src/ /usr/share/nginx/html/

EXPOSE 80
```

5-3) Playwright 설치

```
npm install --save-dev @playwright/test
npx playwright install chromium
```

package.json의 scripts에 한 줄 추가합니다.

```
"test:e2e": "playwright test"
```

?? playwright.config.js

```
import { defineConfig, devices } from "@playwright/test";

export default defineConfig({
  testDir: "./tests/e2e",
  timeout: 30_000,
  fullyParallel: true,
  reporter: [
    ["list"],
    ["html", { open: "never" }],
  ],
  use: {
    baseURL: process.env.BASE_URL || "http://localhost:8080",
    trace: "on-first-retry",
  },
});
```

```
projects: [
  { name: "chromium", use: { ...devices["Desktop Chrome"] } },
],
});
```

5-4) E2E 테스트 작성

tests/e2e/ 디렉터리에 다음 파일을 만듭니다.

?? tests/e2e/calculator.spec.js

```
import { test, expect } from "@playwright/test";

test.describe("계산기 E2E", () => {
  test.beforeEach(async ({ page }) => {
    await page.goto("/");
  });

  test("페이지 제목이 보인다", async ({ page }) => {
    await expect(page.locator("h1")).toHaveText("간단 계산기");
  });

  test("12 + 30 = 42", async ({ page }) => {
    await page.getByTestId("inputA").fill("12");
    await page.getByTestId("inputB").fill("30");
    await page.getByTestId("btn-add").click();
    await expect(page.getByTestId("display")).toHaveText("42");
  });

  test("100 ÷ 4 = 25", async ({ page }) => {
    await page.getByTestId("inputA").fill("100");
    await page.getByTestId("inputB").fill("4");
    await page.getByTestId("btn-divide").click();
    await expect(page.getByTestId("display")).toHaveText("25");
  });

  test("0으로 나누면 에러 메시지가 표시된다", async ({ page }) => {
    await page.getByTestId("inputA").fill("9");
    await page.getByTestId("inputB").fill("0");
    await page.getByTestId("btn-divide").click();
    await expect(page.getByTestId("display")).toContainText("에러");
  });
});
```

로컬에서 한번 돌려보고 싶다면, 한 터미널에서 npm start로 서버를 띄운 뒤 다른 터미널에서:

```
npm run test:e2e
```

5-5) CD 워크플로우 작성

CD는 main에 push될 때만 돌고, 빌드 → E2E → GHCR 푸시 → Pages 배포의 4단계를 거칩니다.

?? `.github/workflows/cd.yml`

```
name: CD

on:
  push:
    branches: [main]

permissions:
  contents: read
  packages: write
  pages: write
  id-token: write

env:
  IMAGE_NAME: ghcr.io/${{ github.repository }}

jobs:
  build-and-e2e:
    name: 1) Docker 빌드 + E2E 테스트
    runs-on: ubuntu-latest
    outputs:
      image-tag: ${{ steps.meta.outputs.tag }}
    steps:
      - uses: actions/checkout@v4
      - id: meta
        run: echo "tag=sha-${GITHUB_SHA::7}" >> "$GITHUB_OUTPUT"
      - run: docker build -t calculator:test .
      - run: docker run -d --rm --name calc -p 8080:80 calculator:test
      - name: 컨테이너가 뜰 때까지 대기
        run: |
          for i in {1..20}; do
            if curl -sf http://localhost:8080 >/dev/null; then echo OK; exit 0; fi
            sleep 1
          done
          exit 1
      - uses: actions/setup-node@v4
        with:
          node-version: "20"
```

```

    cache: "npm"
  - run: npm ci
  - run: npx playwright install --with-deps chromium
  - env:
    BASE_URL: http://localhost:8080
    run: npm run test:e2e
  - if: always()
    run: docker stop calc || true

push-image:
  name: 2) GHCR에 이미지 등록
  runs-on: ubuntu-latest
  needs: build-and-e2e
  steps:
    - uses: actions/checkout@v4
    - uses: docker/login-action@v3
      with:
        registry: ghcr.io
        username: ${ github.actor }
        password: ${ secrets.GITHUB_TOKEN }
    - run: |
        IMAGE=${IMAGE_NAME,,}
        docker build -t $IMAGE:latest -t $IMAGE:${ needs.build-and-e2e.outputs.image-tag } .
        docker push $IMAGE:latest
        docker push $IMAGE:${ needs.build-and-e2e.outputs.image-tag }

deploy-pages:
  name: 3) GitHub Pages 배포
  runs-on: ubuntu-latest
  needs: build-and-e2e
  environment:
    name: github-pages
    url: ${ steps.deploy.outputs.page_url }
  steps:
    - uses: actions/checkout@v4
    - uses: actions/configure-pages@v5
    - uses: actions/upload-pages-artifact@v3
      with:
        path: src
    - id: deploy
      uses: actions/deploy-pages@v4

```

5-6) GitHub Pages 활성화

CD가 Pages로 배포할 수 있도록 설정을 한 번 해줘야 합니다.

- ☐ GitHub 저장소 → Settings → Pages
- ☐ Source 드롭다운에서 "GitHub Actions" 선택

?? 이 설정은 한 번만 하면 됩니다. 이후 CD가 알아서 배포합니다.

5-7) 커밋 → PR

```
git add .
git commit -m "feat: Dockerfile + Playwright E2E + CD 파이프라인"
git push -u origin feature/cd
```

PR을 올리면, 이 브랜치는 cd.yml을 추가했지만 트리거가 main push이므로 PR 단계에선 CD가 돌지 않습니다. CI 4개만 통과하면 됩니다.

5-8) Squash merge → CD 작동 관찰

Squash merge 직후, 저장소의 Actions 탭으로 가면 "CD" 워크플로우가 막 시작된 것이 보입니다.

- 잡 1) Docker 빌드 + E2E 테스트 - 진행 로그를 클릭해서 보세요. 컨테이너 띄우기, Playwright 4개 테스트, 컨테이너 정지의 단계가 순서대로 보입니다.
- 잡 2) GHCR에 이미지 등록 - 끝나면 저장소 메인 화면 우측의 "Packages"에 이미지가 등장합니다.
- 잡 3) GitHub Pages 배포 - 잡 로그 마지막에 page_url이 표시됩니다.

5-9) 배포 결과 직접 확인

- ☐ <https://<USER>.github.io/simple-calculator/> 에 접속 → 본인의 계산기가 보이는지 확인
- ☐ 실제로 $12 + 30$, $100 \div 4$ 같은 연산이 동작하는지 확인
- ☐ GitHub 프로필 → Packages 탭에서 simple-calculator 이미지 확인
- ☐ 이미지 클릭 → 태그 latest와 sha-XXXXXXX가 둘 다 보이는지 확인

STEP 5 체크리스트

- ☐ feature/cd 브랜치에서 작업했다
- ☐ Settings → Pages 에서 Source를 GitHub Actions로 설정했다
- ☐ PR이 CI 통과 후 머지됐다
- ☐ CD가 자동으로 시작되어 3개 잡이 모두 초록색이 됐다
- ☐ GitHub Pages 주소에서 계산기가 동작한다
- ☐ GHCR에 이미지가 등록됐다

GHCR(GitHub Container Registry)의 이미지를 받아 실행하는 절차입니다.

1. 로그인 (private 이미지인 경우만)

public 이미지면 건너뛰어도 됩니다.

GitHub PAT 필요 (scope: read:packages)

```
echo $GITHUB_TOKEN | docker login ghcr.io -u <github-username>
--password-stdin
```

PAT는 <https://github.com/settings/tokens> 에서 [read:packages](#) 권한으로 발급.

2. 이미지 pull

```
docker pull ghcr.io/<owner>/<image>:<tag>
```

예: 이 저장소([ischung/simple-calculator-starter](#))라면

```
docker pull ghcr.io/ischung/simple-calculator-starter:latest
```

태그를 모르면 GitHub 저장소 우측 **Packages** 섹션에서 확인할 수 있습니다.

3. 컨테이너 실행

이 프로젝트의 Dockerfile은 웹 서버(예: 3000번)를 띄우는 구조이므로:

```
docker run --rm -p 3000:80 ghcr.io/ischung/simple-calculator:latest
```

- **--rm**: 종료 시 컨테이너 자동 삭제
- **-p** [호스트포트:컨테이너포트](#): 포트 매핑
- 백그라운드 실행하려면 **-d** 추가

실행 후 브라우저에서 <http://localhost:3000> 으로 접속.

4. 자주 쓰는 보조 명령

```
docker ps          # 실행 중인 컨테이너 확인
```

```
docker logs <container_id> # 로그 확인
docker stop <container_id> # 중지
docker images               # 받아둔 이미지 목록
```

STEP 6 · 일부러 깨뜨려보기

CI/CD가 잘 도는 상황만 보면 그 가치가 잘 안 느껴집니다. 마지막으로 한 번, 일부러 망가진 코드를 PR로 올려서 자동 검사가 막아주는 경험을 합니다. 이 PR은 머지하지 않습니다 - 깨뜨려보는 용도일 뿐이니깐요.

6-1) feature 브랜치

```
git switch main && git pull && git switch -c feature/break
```

6-2) multiply 함수에 버그 심기

src/calculator.js의 multiply 함수에서 곱셈을 덧셈으로 바꿉니다.

```
export function multiply(a, b) {
  return a + b;    // 일부러 잘못 구현
}
```

6-3) PR 올리기

```
git add src/calculator.js
git commit -m "broken: multiply가 더하기를 함 (CI 실패 의도)"
git push -u origin feature/break
```

PR을 올리면 잠시 후 CI가 돌고, unit-test 잡이 빨갛게 x으로 바뀌며 실패합니다.

6-4) 무엇을 관찰할 것인가

- PR 페이지에 빨간 X 아이콘이 뜨고 "All checks have failed" 메시지
- Details 링크 → 어느 테스트가 어떤 값을 기대했고 실제로 뭐가 나왔는지 보여줌
- 브랜치 보호 설정을 했다면 Squash merge 버튼이 비활성화되어 있음
- CD는 main 트리거이므로 아예 돌지 않음 - 망가진 코드가 배포까지 가지 못함

6-5) 정리

이 PR은 머지하지 않습니다. 화면 우측의 "Close pull request" 버튼으로 닫고, 로컬에서도 브랜치를 삭제합니다.

```
git switch main  
git branch -D feature/break      # 로컬에서 강제 삭제  
git push origin --delete feature/break  # 원격에서도 삭제
```

?? "실패한 PR을 머지할 수 없는 경험"이 CI의 가치를 가장 잘 알려줍니다. 이게 사람의 리뷰만으로는 잘 안 잡히는 회귀 버그를 막아주는 메커니즘입니다.

부록 A · 자주 막히는 곳

Q1. CI에서 "permission denied" 오류

A. ci.yml에는 별도 권한 설정이 필요 없지만, cd.yml의 permissions 블록에 packages: write, pages: write, id-token: write가 모두 있어야 합니다. 빠진 게 있으면 추가하고 빈 커밋을 push해서 다시 돌려보세요.

Q2. GHCR push가 "name unknown" 또는 "denied"

A. GHCR 이미지 이름은 모두 소문자여야 합니다. cd.yml에서 IMAGE=\${IMAGE_NAME,,} 부분이 그 처리를 담당하므로, 이 줄을 빠뜨리지 않았는지 확인하세요.

Q3. Pages가 404

A. Settings → Pages에서 Source를 "GitHub Actions"로 바꿨는지 다시 확인. 이 설정은 처음 한 번만 하면 됩니다. 설정 직후 첫 머지에서만 배포가 시작됩니다.

Q4. Playwright가 로컬에서는 통과, CI에선 timeout

A. cd.yml의 "컨테이너가 뜰 때까지 대기" 단계의 반복 횟수(20)를 30~40으로 늘려보세요. 가끔 GitHub Actions 러너가 느릴 때가 있습니다.

Q5. npm audit이 빨강게 뜸

A. package.json scripts의 "audit": "npm audit --audit-level=high" 부분에서 high를 critical로 낮추거나, npm audit fix로 의존성을 갱신해보세요. 단 fix --force는 호환성을 깰 수 있으니 주의.

부록 B · 명령어 cheatsheet

```
# 새 작업 시작
git switch main && git pull
git switch -c feature/<이름>

# 로컬 검증
npm run lint
npm run test:unit
npm run test:integration
npm test                                # 단위 + 통합 한 번에
npm run test:e2e                        # Playwright (npm start 별도 터미널)
npm start                               # http://localhost:8080

# 커밋 → PR
git add .
git commit -m "feat: <설명>"
git push -u origin feature/<이름>

# 머지 후 정리
git switch main && git pull
git branch -d feature/<이름>
```

부록 C · STEP별 도달 상태

STEP	추가/수정 파일	테스트 수	관찰 포인트
0	package.json (npm init)	-	GitHub repo 생성
1	src/* 전체, tests/unit, tests/integration, ESLint/Jest 설정, ci.yml	단위 4 + 통합 2	첫 CI 통과
2	subtract 함수 + 버튼 + 단위 테스트	단위 6 + 통합 2	CI 자동 재실행
3	multiply 함수 + 버튼 + 단위 테스트	단위 8 + 통합 2	CI 통과
4	divide 함수 (0 처리) + 버튼 + 단위/통합 테스트	단위 11 + 통합 3	CI 통과
5	Dockerfile, tests/e2e, playwright.config.js, cd.yml	+ E2E 4	자동 배포!
6	multiply 일부러 깨기 (머지 안 함)	-	CI 실패 → 머지 차단